What is Redux?

Redux is a JavaScript state management library used to manage the global state of an application in a predictable way.

In large applications, many components need access to the same data. Instead of passing data through many components (prop drilling), Redux stores all application state in a central store.

Key Idea:
One centralized store
Components read data from the store
Components update data using actions

Redux is commonly used with React applications.

Why Redux is Used

1. Centralized State Management

All application data is stored in one store.

Example in a MERN project:

- User login data
- Shopping cart items
- Product list

---

2. Avoids Prop Drilling

Without Redux:

App → Dashboard → Profile → Settings → Component

Props must be passed through every component.

Redux allows:

Component → Redux Store

Direct access.

---

3. Predictable State Changes

State changes only through actions and reducers, making debugging easier.

---

4. Easier Debugging

Redux provides tools like Redux DevTools to track state changes.

Redux Architecture

Redux has 4 main parts

Store

Central place where application state is stored.

Example:

```
store = {
  counter: 0,
  user: {},
  products: []
}
```

Action

An action is an object describing what happened.

Example

```
{
  type: "INCREMENT"
}
```

Reducer

A function that updates state based on actions.

Example

```
function reducer(state, action){
   if(action.type === "INCREMENT"){
      return {count: state.count + 1}
   }
}
```

Dispatch

Dispatch sends actions to reducers.

```
dispatch({type:"INCREMENT"})
```

Redux in a MERN Stack Project

MERN =

- MongoDB – Database
- Express.js – Backend framework
- React – Frontend
- Node.js – Runtime

Redux is used in the React frontend to manage data received from the backend API.

Example Flow:

MongoDB → Express API → React → Redux Store → Components

Folder Structure (React + Redux)

```
mern-redux-app
│
├── backend
│    server.js
│
└── frontend
    src
     │
     ├── app
     │    store.js
     │
     ├── features
     │    counterSlice.js
     │
     ├── components
     │    Counter.jsx
     │
     ├── App.jsx
     └── main.jsx
```

Install Redux in React

Inside frontend:

npm install @reduxjs/toolkit react-redux axios

Libraries used:

- Redux Toolkit – Simplified Redux
- React Redux – Connect Redux with React
- Axios – API calls

Backend Code (Node + Express)

server.js

```
const express = require("express");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

let counter = 0;

app.get("/counter", (req,res)=>{
   res.json({value: counter});
});

app.post("/increment", (req,res)=>{
   counter++;
   res.json({value: counter});
});

app.post("/decrement", (req,res)=>{
   counter--;
   res.json({value: counter});
});

app.listen(5000,()=>{
   console.log("Server running on port 5000");
});
```

Run backend:

```
node server.js
```

Redux Store

store.js

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "../features/counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

Redux Slice

counterSlice.js

```
import { createSlice } from "@reduxjs/toolkit";
```

```
const counterSlice = createSlice({
  name: "counter",
  initialState: {
    value: 0
  },

  reducers: {
    increment: (state) => {
      state.value += 1;
    },

    decrement: (state) => {
      state.value -= 1;
    }
  }
});

export const { increment, decrement } = counterSlice.actions;
export default counterSlice.reducer;
```

Connect Redux with React

main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

import { Provider } from "react-redux";
import { store } from "./app/store";

ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
      <App />
  </Provider>
);
```

Provider allows React components to access Redux store.

React Component

Counter.jsx

```
import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { increment, decrement } from "../features/counterSlice";

function Counter(){

   const count = useSelector((state)=>state.counter.value);
   const dispatch = useDispatch();
```

```
  return(
    <div>
      <h2>Counter: {count}</h2>

      <button onClick={()=>dispatch(increment())}>
        Increment
      </button>

      <button onClick={()=>dispatch(decrement())}>
        Decrement
      </button>
    </div>
  )
}

export default Counter;
```

App.jsx

```
import Counter from "./components/Counter";

function App() {
  return (
    <div>
      <h1>MERN Redux Counter App</h1>
      <Counter/>
    </div>
  );
}

export default App;
```

How Data Flows in Redux

User clicks button
↓
Dispatch Action
↓
Reducer updates state
↓
Store updates
↓
React component re-renders

Alternatives to Redux

React Context API

Built-in React state sharing.

Best for:

- Small applications

Zustand

Very simple state management.

Example:

```
const useStore = create((set)=>({
  count:0,
  increment:()=>set(state=>({count:state.count+1}))
}))
```

Recoil

State management developed by Meta Platforms.

Good for complex React apps.

MobX

Uses reactive programming instead of reducers.

When to Use Redux

Use Redux when:

✔ Large applications
✔ Many components share data
✔ Complex state logic
✔ Debugging required